

Backpropagation e discesa del gradiente

Dal singolo esempio al dataset, dagli indici alla forma matriciale

Questa scheda introduce gli algoritmi di backpropagation e di discesa del gradiente utilizzati nell'allenamento di reti neurali, con particolare riferimento alla loro implementazione nell'applicazione [Dentro la rete](#).

Durante l'allenamento di una rete neurale, pesi e bias vengono modificati con l'obiettivo di ridurre l'errore E . Per un generico parametro p , la discesa del gradiente usa la derivata $\partial E / \partial p$ per stabilire l'aggiornamento:

$$p_{\text{nuovo}} = p - \eta \frac{\partial E}{\partial p}, \quad (1)$$

dove la derivata è calcolata al valore corrente p del parametro; η viene detto tasso di apprendimento. Nelle reti *feed-forward* — in cui le connessioni procedono dagli input verso gli strati successivi senza formare cicli — il metodo standard usato a questo scopo è la *backpropagation*. Come vedremo, essa applica la regola della catena dall'uscita verso gli strati precedenti riutilizzando, grazie a una formulazione ricorsiva, derivate già calcolate.

La procedura descritta è anche quella che l'applicazione rende osservabile durante **Un dato alla volta**. Il calcolo sarà sviluppato dapprima con gli indici, poi collegato all'allenamento sull'intero dataset e infine raccolto in forma vettoriale e matriciale.

1. Notazione e significato dei simboli

Numeriamo gli strati da 0 a L . Lo strato 0 contiene gli input; lo strato L è lo strato di uscita. Indichiamo con n_ℓ il numero di nodi dello strato ℓ ; i nodi dello strato ℓ sono numerati da 1 a n_ℓ .

Indichiamo con $x = (x_1, \dots, x_{n_0})^T$ il vettore colonna che raccoglie gli input del singolo esempio e poniamo $a_i^{(0)} = x_i$. Useremo inoltre la seguente convenzione:

Simbolo	Significato
$a_i^{(\ell)}$	valore del nodo i dello strato ℓ ; per $\ell = 0$, $a_i^{(0)} = x_i$
$w_{ij}^{(\ell)}$	peso della connessione dal nodo j dello strato $\ell - 1$ al neurone i dello strato ℓ
$b_i^{(\ell)}$	bias del neurone i dello strato ℓ
$z_i^{(\ell)}$	valore pre-attivazione del neurone i dello strato ℓ
f_ℓ	funzione di attivazione usata nello strato ℓ
E	errore (<i>loss</i>) del singolo esempio osservato
$\delta_i^{(\ell)}$	derivata $\partial E / \partial z_i^{(\ell)}$
η	tasso di apprendimento

Nell'equazione (1), p indica quindi il valore corrente di uno qualsiasi dei parametri $w_{ij}^{(\ell)}$ o $b_i^{(\ell)}$ della rete.

Per $a_i^{(\ell)}$ è ammesso anche $\ell = 0$; per $w_{ij}^{(\ell)}$, $b_i^{(\ell)}$, $z_i^{(\ell)}$, f_ℓ e $\delta_i^{(\ell)}$ si intende $\ell = 1, \dots, L$. Quando consideriamo il passaggio dallo strato $\ell - 1$ allo strato ℓ , si intende quindi $\ell = 1, \dots, L$.

Nei problemi con una sola uscita, y indica il valore bersaglio; nel caso multiclasse c indica l'indice della classe corretta e t_i le componenti del bersaglio *one-hot* (1 per la classe corretta, 0 per tutte le altre).

Collegamento con il codice dell'applicazione

La struttura degli array usati nel nucleo dell'applicazione corrisponde direttamente alla notazione matematica. Tuttavia, nella scheda gli indici dei neuroni partono da 1, mentre gli array JavaScript partono da 0; perciò nella tabella gli indici dei neuroni compaiono come $i-1$ e $j-1$.

Matematica	Array nell'applicazione
$w_{ij}^{(\ell)}$	<code>weights[l-1][i-1][j-1]</code>
$b_i^{(\ell)}$	<code>bias[l-1][i-1]</code>
$z_i^{(\ell)}$	<code>zs[l][i-1]</code>
$a_i^{(\ell)}$	<code>as[l][i-1]</code>
$\delta_i^{(\ell)}$	<code>delta[l][i-1]</code>

In particolare, `as[0]` contiene gli input; `zs[l]` e `delta[l]` corrispondono agli strati $\ell = 1, \dots, L$. Gli array `weights[l-1]` e `bias[l-1]` rappresentano invece il passaggio dallo strato $\ell - 1$ allo strato ℓ .

2. Forward pass: calcolo dell'uscita in notazione a indici

Per $\ell = 1, \dots, L$, il neurone i dello strato ℓ riceve i valori $a_j^{(\ell-1)}$ da tutti i nodi dello strato precedente, ne calcola prima la somma pesata

$$z_i^{(\ell)} = \sum_{j=1}^{n_{\ell-1}} w_{ij}^{(\ell)} a_j^{(\ell-1)} + b_i^{(\ell)} \quad (2)$$

e, negli strati in cui l'attivazione è applicata componente per componente, applica poi la funzione di attivazione:

$$a_i^{(\ell)} = f_\ell(z_i^{(\ell)}). \quad (3)$$

Nel caso multiclasse dell'applicazione, nello strato di uscita viene invece applicata la softmax all'intero vettore dei valori pre-attivazione. Il *forward pass*, cioè il calcolo dell'uscita della rete a partire dagli input, ripete queste due operazioni strato dopo strato fino all'uscita. Per un problema con una sola uscita, quest'ultima è

$$\hat{y} = a_1^{(L)}.$$

Nell'applicazione, per la classificazione binaria e per la regressione come errore del singolo esempio si considera l'errore quadratico

$$E = \frac{1}{2} (\hat{y} - y)^2.$$

Nella classificazione multiclasse, l'uscita è un vettore di probabilità prodotto dalla softmax e l'errore è l'entropia incrociata

$$E = -\ln p_c.$$

3. Il termine δ

Come abbiamo visto nell'equazione (1), per aggiornare pesi e bias mediante la discesa del gradiente, servono le derivate parziali

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} \quad \text{e} \quad \frac{\partial E}{\partial b_i^{(\ell)}}.$$

Per organizzare questo calcolo, associamo a ogni neurone una quantità che verrà riutilizzata nel calcolo a ritroso delle derivate (*backward pass*):

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial z_i^{(\ell)}}.$$

Il simbolo $\delta_i^{(\ell)}$, spesso chiamato informalmente **segnale d'errore**, rappresenta quindi la derivata dell'errore E rispetto al valore pre-attivazione $z_i^{(\ell)}$ del neurone corrispondente; esprime quindi la sensibilità locale, con segno, dell'errore rispetto a variazioni della pre-attivazione.

Quando l'attivazione è applicata componente per componente, poiché

$$a_i^{(\ell)} = f_\ell(z_i^{(\ell)}),$$

la regola della catena dà anche

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial a_i^{(\ell)}} f'_\ell(z_i^{(\ell)}).$$

Nel paragrafo seguente useremo $\delta_i^{(\ell)}$ per ricavare le derivate rispetto ai pesi e al bias che entrano nel neurone.

4. Derivata rispetto a un peso e a un bias

Consideriamo un peso preciso, $w_{ij}^{(\ell)}$. Nella somma

$$z_i^{(\ell)} = \sum_{q=1}^{n_{\ell-1}} w_{iq}^{(\ell)} a_q^{(\ell-1)} + b_i^{(\ell)},$$

quel peso compare soltanto nel termine $w_{ij}^{(\ell)} a_j^{(\ell-1)}$. Quindi

$$\frac{\partial z_i^{(\ell)}}{\partial w_{ij}^{(\ell)}} = a_j^{(\ell-1)}.$$

Applicando la regola della catena:

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} = \frac{\partial E}{\partial z_i^{(\ell)}} \frac{\partial z_i^{(\ell)}}{\partial w_{ij}^{(\ell)}} = \delta_i^{(\ell)} a_j^{(\ell-1)}. \quad (4)$$

Per il bias,

$$\frac{\partial z_i^{(\ell)}}{\partial b_i^{(\ell)}} = 1,$$

e dunque

$$\frac{\partial E}{\partial b_i^{(\ell)}} = \delta_i^{(\ell)}. \quad (5)$$

Le due relazioni si possono quindi riassumere nella forma:

$$\frac{\partial E}{\partial w} = \delta_{\text{arrivo}} a_{\text{partenza}}, \quad \frac{\partial E}{\partial b} = \delta_{\text{neurone}}.$$

5. Il termine δ nello strato di uscita

I valori di $\delta_i^{(L)}$ per i neuroni in uscita si ricavano analiticamente dalla funzione di errore e dalla trasformazione di uscita: in questo paragrafo vedremo in dettaglio i casi implementati nell'applicazione. Negli strati nascosti, invece, i valori di $\delta_i^{(\ell)}$ vengono calcolati ricorsivamente a partire da quelli dello strato successivo, come verrà descritto nel §6.

5.1 Classificazione binaria nell'applicazione: sigmoide + errore quadratico

Con una sola uscita, utilizzando sigmoide ed errore quadratico come nell'applicazione, si ha

$$a = \hat{y} = \sigma(z), \quad E = \frac{1}{2}(a - y)^2.$$

Poiché

$$\frac{\partial E}{\partial a} = a - y \quad \text{e} \quad \sigma'(z) = a(1 - a),$$

si ottiene

$$\delta^{(L)} = (a - y) a(1 - a). \quad (6a)$$

Nella pratica, tuttavia, per la classificazione binaria è comune associare la sigmoide all'entropia incrociata binaria; in quel caso il termine di uscita si semplifica in $\delta = \hat{y} - y$.

5.2 Regressione nell'applicazione: uscita lineare + errore quadratico

Se l'ultimo strato usa $a = z$, allora $f'(z) = 1$ e quindi

$$\delta^{(L)} = a - y. \quad (6b)$$

5.3 Classificazione multiclasse: softmax + entropia incrociata

Se $z_i^{(L)}$ è il valore pre-attivazione del neurone di uscita i , la softmax produce

$$p_i = \frac{e^{z_i^{(L)}}}{\sum_{k=1}^{n_L} e^{z_k^{(L)}}}.$$

Utilizziamo la codifica one-hot, in cui indichiamo con t_i il valore bersaglio: $t_i = 1$ per la classe corretta e $t_i = 0$ per tutte le altre: l'entropia incrociata (*cross-entropy*) del singolo esempio è

$$E = - \sum_{q=1}^{n_L} t_q \ln p_q = - \ln p_c,$$

dove c è la classe corretta. Poiché

$$\ln p_q = z_q^{(L)} - \ln \left(\sum_{k=1}^{n_L} e^{z_k^{(L)}} \right)$$

e $\sum_i t_i = 1$, possiamo riscrivere

$$E = - \sum_{q=1}^{n_L} t_q z_q^{(L)} + \ln \left(\sum_{k=1}^{n_L} e^{z_k^{(L)}} \right).$$

Derivando rispetto a $z_i^{(L)}$ si ottiene quindi

$$\delta_i^{(L)} = \frac{\partial E}{\partial z_i^{(L)}} = -t_i + \frac{e^{z_i^{(L)}}}{\sum_{k=1}^{n_L} e^{z_k^{(L)}}} = p_i - t_i. \quad (6c)$$

6. Il termine δ in uno strato nascosto

Consideriamo il neurone i di uno strato nascosto ℓ . La sua attivazione $a_i^{(\ell)}$ raggiunge tutti i neuroni $k = 1, \dots, n_{\ell+1}$ dello strato successivo. Per ciascuno di essi,

$$z_k^{(\ell+1)} = \sum_{q=1}^{n_\ell} w_{kq}^{(\ell+1)} a_q^{(\ell)} + b_k^{(\ell+1)}.$$

Vogliamo calcolare

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial z_i^{(\ell)}}.$$

L'errore dipende da $z_i^{(\ell)}$ attraverso l'attivazione $a_i^{(\ell)}$ e poi attraverso tutti i neuroni successivi. La regola della catena deve quindi sommare tutti questi percorsi:

$$\frac{\partial E}{\partial a_i^{(\ell)}} = \sum_{k=1}^{n_{\ell+1}} \frac{\partial E}{\partial z_k^{(\ell+1)}} \frac{\partial z_k^{(\ell+1)}}{\partial a_i^{(\ell)}}.$$

Ma

$$\frac{\partial E}{\partial z_k^{(\ell+1)}} = \delta_k^{(\ell+1)} \quad \text{e} \quad \frac{\partial z_k^{(\ell+1)}}{\partial a_i^{(\ell)}} = w_{ki}^{(\ell+1)},$$

perciò

$$\frac{\partial E}{\partial a_i^{(\ell)}} = \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)} \delta_k^{(\ell+1)}.$$

Infine, $a_i^{(\ell)} = f_\ell(z_i^{(\ell)})$, quindi

$$\delta_i^{(\ell)} = f'_\ell(z_i^{(\ell)}) \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)} \delta_k^{(\ell+1)}. \quad (7)$$

Questa formula ricorsiva permette quindi di calcolare il termine δ di uno strato nascosto a partire dai termini δ dello strato successivo.

In forma mnemonica, la struttura della ricorrenza è quindi:

$$\delta_{\text{neurone}} = f'(z_{\text{neurone}}) \cdot \sum_{\text{successivi}} (w \cdot \delta_{\text{successivo}}).$$

7. Backpropagation per un singolo esempio

Per un singolo esempio, fissati input e bersaglio:

1. **Forward.** Si calcolano e si conservano tutti gli $z_i^{(\ell)}$ e gli $a_i^{(\ell)}$ mediante (2) e (3).
2. **Errore.** Si calcola E a partire dall'uscita della rete e dal bersaglio.
3. **Delta di uscita.** Si calcolano i $\delta_i^{(L)}$ in base alla funzione di errore e alla trasformazione di uscita. Nei casi trattati dall'applicazione si utilizza (6a), (6b) o (6c).
4. **Delta nascosti.** Si calcolano i $\delta_i^{(\ell)}$ dagli ultimi strati verso i primi applicando ricorsivamente (7).
5. **Derivate dei parametri.** Noti tutti i termini $\delta_i^{(\ell)}$, le derivate di E rispetto a pesi e bias si ottengono con (4) e (5).
6. **Aggiornamento.** Si ricavano le variazioni dei parametri mediante (1) e si applicano insieme.

Tutte le derivate devono riferirsi allo stesso stato della rete: per questo si calcolano prima le derivate di E rispetto a tutti i pesi e bias e soltanto dopo si applicano insieme gli aggiornamenti.

8. Dal singolo esempio al dataset: SGD e mini-batch

Finora abbiamo calcolato le derivate dell'errore $E_r(\theta)$ prodotto da un singolo esempio; tuttavia, l'obiettivo dell'allenamento è minimizzare l'errore medio sull'intero insieme di N esempi di allenamento. Indicando con θ l'insieme di tutti i pesi e bias della rete e con $\bar{E}(\theta)$ l'errore medio, abbiamo:

$$\bar{E}(\theta) = \frac{1}{N} \sum_{r=1}^N E_r(\theta).$$

Per linearità della derivazione, il gradiente dell'errore medio rispetto ai parametri è la media dei gradienti $\nabla E_r(\theta)$ degli errori associati ai singoli esempi:

$$\nabla \bar{E}(\theta) = \frac{1}{N} \sum_{r=1}^N \nabla E_r(\theta).$$

Gradient descent sull'intero dataset. Si calcolano, nello stesso stato della rete, i gradienti degli errori associati a tutti gli esempi del dataset, se ne fa la media e si applica un solo aggiornamento:

$$\theta_{\text{nuovo}} = \theta - \eta \nabla \bar{E}(\theta).$$

Questa procedura è detta *batch gradient descent*.

Discesa stocastica del gradiente (SGD, *stochastic gradient descent*). È detta stocastica perché a ogni passo la direzione di aggiornamento dipende dalla scelta casuale di un esempio, anziché essere calcolata usando contemporaneamente l'intero dataset. Nel caso del campionamento indipendente, indichiamo con R l'indice

dell'esempio estratto a un dato passo: a ogni passo R viene scelto uniformemente e indipendentemente tra $1, \dots, N$, e i parametri vengono aggiornati subito:

$$\theta_{\text{nuovo}} = \theta - \eta \nabla E_R(\theta).$$

Poiché ciascun indice ha probabilità $1/N$, a parametri fissati il valore atteso del gradiente $\nabla E_R(\theta)$ è:

$$\mathbb{E}_R[\nabla E_R(\theta)] = \sum_{r=1}^N \frac{1}{N} \nabla E_r(\theta) = \frac{1}{N} \sum_{r=1}^N \nabla E_r(\theta) = \nabla \bar{E}(\theta).$$

Il valore atteso è qui calcolato rispetto alla scelta casuale di R tra gli N esempi del dataset fissato. In questo senso, il gradiente dell'errore associato all'esempio estratto è uno stimatore non distorto del gradiente dell'errore medio. Di conseguenza, a parametri fissati, l'aggiornamento medio di un passo di SGD coincide con quello di un passo di *batch gradient descent* con lo stesso tasso η , anche se il singolo aggiornamento può essere diverso.

Random reshuffling. All'inizio di ogni giro si genera un nuovo ordine casuale $r_1, \dots, r_N$ degli esempi e poi li si utilizza tutti, una volta ciascuno: il campionamento avviene quindi senza reinserimento. È questo l'algoritmo adottato dall'applicazione.

Il *random reshuffling* — cioè rimescolare il dataset a ogni epoca e percorrerlo senza reinserimento — è comune nella pratica.

Se durante il giro i parametri restassero fissi al valore iniziale θ_0 , la media dei gradienti sarebbe esattamente il gradiente dell'errore medio, qualunque sia l'ordine degli esempi:

$$\frac{1}{N} \sum_{k=1}^N \nabla E_{r_k}(\theta_0) = \frac{1}{N} \sum_{r=1}^N \nabla E_r(\theta_0) = \nabla \bar{E}(\theta_0).$$

Poiché ogni permutazione contiene ciascun esempio esattamente una volta, sotto questa ipotesi la media dei gradienti del giro coincide esattamente col gradiente medio del dataset.

Invece, nel *random reshuffling* con aggiornamento dopo ogni esempio, come nell'applicazione, i parametri cambiano durante il giro. Se θ_k indica il valore dei parametri dopo il k -esimo aggiornamento, allora

$$\theta_k = \theta_{k-1} - \eta \nabla E_{r_k}(\theta_{k-1}).$$

Definiamo il gradiente medio effettivamente usato durante il giro:

$$g_{\text{giro}} = \frac{1}{N} \sum_{k=1}^N \nabla E_{r_k}(\theta_{k-1}).$$

Per confrontarlo con $\nabla \bar{E}(\theta_0)$ osserviamo anzitutto che, dopo $k-1$ aggiornamenti,

$$\theta_{k-1} - \theta_0 = -\eta \sum_{s=1}^{k-1} \nabla E_{r_s}(\theta_{s-1}).$$

A numero di esempi fissato, se i gradienti restano limitati nell'intorno percorso dai parametri, la somma contiene un numero finito di termini e quindi

$$\theta_{k-1} - \theta_0 = O(\eta).$$

Se inoltre il gradiente di ciascun E_r varia in modo sufficientemente regolare con i parametri — per esempio è localmente lipschitziano — una variazione $O(\eta)$ dei parametri produce una variazione $O(\eta)$ anche del gradiente:

$$\nabla E_{r_k}(\theta_{k-1}) = \nabla E_{r_k}(\theta_0) + O(\eta).$$

Mediando sugli N aggiornamenti del giro, sempre a N fissato, si ottiene

$$g_{\text{giro}} = \frac{1}{N} \sum_{k=1}^N \nabla E_{r_k}(\theta_0) + O(\eta) = \nabla \bar{E}(\theta_0) + O(\eta).$$

Quindi, il gradiente medio effettivamente usato durante un giro differisce dal gradiente dell'errore medio calcolato all'inizio del giro di una quantità di ordine $O(\eta)$.

L'aggiornamento complessivo del giro è

$$\theta_N = \theta_0 - N\eta g_{\text{giro}} = \theta_0 - N\eta \nabla \bar{E}(\theta_0) + O(\eta^2).$$

Lo spostamento principale dei parametri è di ordine η , mentre la differenza rispetto al corrispondente passo batch è di ordine η^2 . Pertanto, con η sufficientemente piccolo, un giro completo di random reshuffling con aggiornamento dopo ogni esempio ha, al primo ordine, lo stesso aggiornamento di un passo di batch gradient descent sull'errore medio con tasso di apprendimento $N\eta$. Il fattore N compare perché durante un giro si compiono N aggiornamenti di ampiezza η , mentre il batch gradient descent usa il gradiente medio. Il rimescolamento e gli aggiornamenti intermedi determinano le correzioni di ordine superiore e quindi la traiettoria esatta. Tutti gli aggiornamenti agiscono quindi sullo stesso vettore di parametri: ciascun esempio viene elaborato nello stato della rete lasciato dagli aggiornamenti precedenti.

Mini-batch. Nella pratica si usa spesso una soluzione intermedia tra il batch gradient descent e l'aggiornamento su un singolo esempio. Per un gruppo B di m esempi si calcola

$$g_B(\theta) = \frac{1}{m} \sum_{r \in B} \nabla E_r(\theta)$$

e si aggiorna la rete usando $g_B(\theta)$. Se B è scelto uniformemente a parametri fissati, $g_B(\theta)$ è uno stimatore non distorto del gradiente dell'errore medio. Poiché media i contributi di più esempi, questa quantità varia in genere meno da un campionamento all'altro rispetto al gradiente dell'errore associato a un singolo esempio: costituisce quindi una stima meno rumorosa del gradiente dell'errore medio sull'intero dataset. Rispetto al batch completo, richiede meno calcolo per ciascun aggiornamento e permette di sfruttare efficacemente il calcolo parallelo. Nella pratica i mini-batch sono spesso ottenuti rimescolando il dataset all'inizio dell'epoca, dividendolo in gruppi e aggiornando i parametri dopo ciascun gruppo. L'aggiornamento su un singolo esempio corrisponde al caso $m = 1$; il batch completo corrisponde al caso $m = N$. L'applicazione usa $m = 1$ insieme al random reshuffling senza reinserimento durante l'epoca.

9. Dalle formule a indici alla forma matriciale

Le formule a indici si raccolgono in vettori e matrici scrivendo simultaneamente le stesse equazioni per tutti i neuroni di uno strato.

Per uno strato $\ell = 1, \dots, L$ definiamo:

- $z^{(\ell)} \in \mathbb{R}^{n_\ell}$ è il vettore colonna dei valori pre-attivazione.
- $a^{(\ell)} \in \mathbb{R}^{n_\ell}$ è il vettore colonna delle attivazioni in uscita.
- $b^{(\ell)} \in \mathbb{R}^{n_\ell}$ è il vettore colonna dei bias.

• $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ è la matrice dei pesi. La riga i -esima contiene i pesi delle connessioni che arrivano al neurone i dello strato ℓ ; la colonna j -esima contiene i pesi delle connessioni che partono dal nodo j dello strato $\ell - 1$. In particolare, $(W^{(\ell)})_{ij} = w_{ij}^{(\ell)}$.

Forward

$$\begin{aligned} z^{(\ell)} &= W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \\ a^{(\ell)} &= f_\ell(z^{(\ell)}). \end{aligned}$$

Nell'applicazione la funzione f_ℓ è applicata componente per componente, salvo il caso della softmax nello strato di uscita.

Backward negli strati nascosti

Per uno strato nascosto ($1 \leq \ell \leq L - 1$), raccogliamo in un vettore colonna i termini $\delta_i^{(\ell)}$:

$$\delta^{(\ell)} = \begin{pmatrix} \delta_1^{(\ell)} \\ \vdots \\ \delta_{n_\ell}^{(\ell)} \end{pmatrix} \in \mathbb{R}^{n_\ell}.$$

Per la componente i -esima, la formula ricavata nel §6 è

$$\delta_i^{(\ell)} = f'_\ell(z_i^{(\ell)}) \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)} \delta_k^{(\ell+1)}.$$

La somma a destra è la componente i -esima del prodotto tra matrici $(W^{(\ell+1)})^T \delta^{(\ell+1)}$:

$$\left[(W^{(\ell+1)})^T \delta^{(\ell+1)} \right]_{i,1} = \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)} \delta_k^{(\ell+1)}$$

Scrivendo contemporaneamente tutte le componenti si ottiene quindi

$$\delta^{(\ell)} = \left((W^{(\ell+1)})^T \delta^{(\ell+1)} \right) \odot f'_\ell(z^{(\ell)}).$$

dove $\odot$ indica il prodotto componente per componente, detto anche prodotto di Hadamard.

Derivate di E rispetto ai parametri

Indichiamo con $\frac{\partial E}{\partial W^{(\ell)}}$ la matrice $n_\ell \times n_{\ell-1}$ definita da

$$\left[\frac{\partial E}{\partial W^{(\ell)}} \right]_{ij} = \frac{\partial E}{\partial w_{ij}^{(\ell)}}$$

Per l'equazione (4) abbiamo

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} = \delta_i^{(\ell)} a_j^{(\ell-1)}.$$

Il secondo membro dell'equazione (4) è l'elemento in posizione (i, j) del prodotto tra la matrice colonna $\delta^{(\ell)}$ e la matrice riga $(a^{(\ell-1)})^T$. Possiamo quindi raccogliere tutte le derivate di E rispetto ai pesi nella matrice:

$$\frac{\partial E}{\partial W^{(\ell)}} = \delta^{(\ell)} (a^{(\ell-1)})^T.$$

Per i bias, definito $\frac{\partial E}{\partial b^{(\ell)}}$ come il vettore colonna di componenti $\partial E / \partial b_i^{(\ell)}$, dall'equazione (5), otteniamo l'uguaglianza vettoriale

$$\frac{\partial E}{\partial b^{(\ell)}} = \delta^{(\ell)}.$$

Estensione ai mini-batch. Fin qui $z^{(\ell)}$, $a^{(\ell)}$ e $\delta^{(\ell)}$ sono vettori relativi a un singolo esempio. Con un mini-batch, i vettori corrispondenti ai diversi esempi possono essere affiancati come colonne di matrici. Forward, backward e calcolo dei gradienti diventano allora in larga parte operazioni tra matrici, particolarmente adatte all'esecuzione parallela su GPU.